

GATE-ML Technical Design Decisions

GATE-ML Falsification-Audit — Technical Design Documents (ADR log)

ADR-001 — Training framework: PyTorch, not NumPy

ADR-002 — Compute scale: toy-scale on RTX 5050/Colab/Kaggle, not 125-270M on A100

ADR-003 — Cross-mechanism/cross-condition execution: one Mechanism protocol, not per-mechanism or per-condition code paths

ADR-004 — Configuration system: plain YAML + dataclasses, not a config framework

ADR-005 — MI-estimator null construction: circular-shift, not full-shuffle

ADR-006 — Positive-control mechanism: curriculum-ordering, not MoE-routing

ADR-007 — Core mechanism list: 4 (GATE-ML, RigL, GrokFast, curriculum-ordering), not 6

ADR-008 — Data-provenance reporting gate: hard-coded exclusion, not a documented convention

ADR-009 — Logging: local JSONL as source of truth, external dashboard optional

GATE-ML Falsification-Audit — Technical Design Documents (ADR log)

One entry per significant technical decision made across this project. Each follows the same template. Status reflects actual confirmation state, not aspiration — two entries below are marked Proposed rather than Accepted because they were recommended in prior documents but never explicitly confirmed by the user or team; that distinction is deliberate, not an oversight.

ADR-001 — Training framework: PyTorch, not NumPy

Status: Accepted (2026-09-26)

Context: A disk- and network-constrained implementation sandbox couldn't install PyTorch (the default install pulls CUDA-enabled dependencies) and had no route to real corpora. The response was a NumPy-only reimplementing using synthetic in-code token sequences, which was drifting toward becoming the project's actual reported pipeline rather than a throwaway test.

Options considered: - **A.** NumPy-only throughout, with hand-written forward/backward passes for a toy language model. - **B.** Two parallel implementations — NumPy for

constrained sandboxes, PyTorch for real execution. - **C.** PyTorch throughout, using a CPU-only wheel in constrained sandboxes.

Choice: C.

Reason: NumPy has no GPU/CUDA path — adopting it as canonical abandons the RTX 5050 entirely and undermines the whole toy-scale compute plan built around having a GPU available. A default `pip install torch` pulls several large `nvidia-*` dependency packages that most likely caused the original disk-budget failure; the dedicated CPU-only wheel (`pip install torch --index-url https://download.pytorch.org/whl/cpu`) skips them entirely. The RTX 5050 laptop itself was confirmed to have normal network access and disk space — the constraint was local to one sandbox, not the real execution environment. Option B was rejected because two parallel implementations of training/mechanism/battery logic is a structural drift risk, which is precisely the class of problem this project's own falsification logic exists to catch.

Consequences: Constrained sandboxes run slower (CPU-only) for dev/wiring tests — acceptable, since those tests use synthetic data and tiny toy models anyway. The previously-open question of hand-writing forward/backward passes vs. accepting a simplified proxy model is moot: autograd handles differentiation, and a proxy model was never going to be acceptable regardless of framework (no genuine learning means no real effect for the battery to falsify).

Revisit when: A future sandbox can't fit even the CPU-only wheel, or a genuine requirement emerges for a PyTorch-free deployment target. Neither has happened.

ADR-002 — Compute scale: toy-scale on RTX 5050/Colab/Kaggle, not 125-270M on A100

Status: Accepted

Context: The original proposal and the falsification-audit pivot budgeted ~486-650 GPU-hours across 54 runs at 125-270M parameter scale, assuming 4×A100 access. That access never materialized; only a single RTX 5050 laptop (8GB VRAM) plus free Colab/Kaggle tiers are available.

Options considered: - **A.** Wait for A100 access before starting real experiments. - **B.** Attempt the original 125-270M scale on the RTX 5050 regardless. - **C.** Rescope to 10-25M parameters, shorter runs, and fewer mechanisms.

Choice: C.

Reason: A has no confirmed timeline and risks the 16-week project stalling indefinitely on a dependency outside the team's control. B fails outright, not just slowly: bytes-per-parameter memory accounting (weights + gradients + Adam's two moment buffers ≈ 16 bytes/param at fp32) already consumes roughly half the 8GB budget before a single activation tensor is allocated at 125M+ scale — an allocation failure, not a slowdown. C is a real, storable scope decision with precedent (GrokFast's own original paper is already toy-scale), not a hidden compromise.

Consequences: Results can't directly claim to generalize to the mechanisms' original, larger-scale evaluations — stated as an explicit limitation. One optional 125M scale-sensitivity data point (GATE-ML only) is budgeted as a stretch check, not a requirement.

Revisit when: Real GPU access beyond the RTX 5050/Colab/Kaggle materializes, with enough of the 16-week timeline left to redo the grid at larger scale.

ADR-003 — Cross-mechanism/cross-condition execution: one Mechanism protocol, not per-mechanism or per-condition code paths

Status: Accepted

Context: The falsification battery requires that Real, Step-matched, and Scrambled conditions be compared fairly across four structurally different mechanisms. If the code implementing each condition or mechanism differs in ways unrelated to the thing actually being tested, the comparison isn't trustworthy regardless of what the results show.

Options considered: - **A.** One training script per mechanism, with condition logic branched inside each. - **B.** One training script per (mechanism, condition) pair. - **C.** A single Mechanism protocol (`signal` / `should_intervene` / `apply`) implemented identically by every mechanism and every condition-wrapper, consumed by one mechanism-agnostic training loop.

Choice: C.

Reason: A and B both risk silent drift — different logging cadence, different checkpoint timing, subtly different optimizer setup between scripts — that could invalidate the fair-comparison claim without anyone noticing until results are already in hand. C makes fairness an architectural invariant rather than a discipline someone has to maintain: the training loop has no code path by which it could treat conditions or mechanisms differently, because it has no way to distinguish them.

Consequences: Every mechanism has to be expressed through a 3-method interface, which took real design work (e.g., mapping curriculum-ordering's per-example difficulty proxy onto a protocol originally shaped around per-step training signals). Adding a 5th mechanism later is a registry entry, not a new script.

Revisit when: A future mechanism's structure genuinely can't be expressed as `signal/should_intervene/apply` — hasn't happened for any of the four core mechanisms or the three stretch candidates.

ADR-004 — Configuration system: plain YAML + dataclasses, not a config framework

Status: Accepted

Context: The project runs across at least three uncoordinated environments (RTX 5050 laptop, Colab, Kaggle, plus occasional constrained dev sandboxes), with two people who need to reproduce each other's runs exactly.

Options considered: - **A.** A config framework (Hydra, OmegaConf). - **B.** Plain dictionaries/argparse, no composition. - **C.** YAML + dataclasses with a small hand-written merge utility (base → mechanism → condition → CLI override).

Choice: C.

Reason: A adds a dependency that must be identically versioned across three-plus environments the team doesn't fully control, and becomes a black box to debug on a free-tier notebook if something breaks. B has no composition discipline, making "no hyperparameter set programmatically, only via config" unenforceable. C gives the same composability with ~40 lines of code the team fully understands and can debug themselves.

Consequences: Some config-merging edge cases (e.g., list-valued overrides) have to be hand-implemented rather than inherited from a mature library — accepted as a small, known cost.

Revisit when: The config surface grows enough (many more mechanisms, nested sweep configs) that the hand-written merge utility becomes its own maintenance burden.

ADR-005 — MI-estimator null construction: circular-shift, not full-shuffle

Status: Accepted

Context: The falsification battery's MI-with-step estimator needs a null distribution to test significance against. AuditBench (synthetic ground-truth validation) found the original full-shuffle null produced a 100% false-positive rate on genuinely step-independent, autocorrelated signals — the exact shape of every real signal this project audits.

Options considered: - **A.** Full-shuffle null (original). - **B.** Circular-shift null (rotate the signal rather than shuffling it). - **C.** Block-permutation null (shuffle contiguous blocks).

Choice: B.

Reason: A destroys the signal's autocorrelation in the null while real data retains it, inflating observed MI from serial correlation alone — measured 100% false-positive rate on a genuinely independent synthetic signal. B preserves autocorrelation in both observed and null data, dropping the false-positive rate to ~5% (correctly calibrated for a nominal $\alpha=0.05$ test). C was tested and found to invert the trade-off — strong against monotonic-ramp step-proxies, weak against the same autocorrelation false-positive B fixes — so no option dominates on both axes, but B is the safer default given every real signal here is smooth by construction.

Consequences: MI-alone detection power against monotonic-ramp-shaped step proxies specifically drops from ~100% to ~50-55% under B — a quantified, documented cost, not a free win. This is exactly why the battery never relies on MI alone; the three-condition CI-based comparison is the primary safeguard, MI a secondary diagnostic.

Revisit when: A future signal shape is found where B's specific weakness (ramp-like proxies) dominates in practice — would justify running both nulls and reporting the disagreement rather than picking one default.

ADR-006 — Positive-control mechanism: curriculum-ordering, not MoE-routing

Status: Proposed — recommended in `Publication_Experiment_Design.md §1`, not yet explicitly confirmed by the user/team.

Context: The novelty verification found the battery needs at least one mechanism from a category with an already-known falsification result, so the protocol can be shown to correctly reproduce that known result before being trusted on genuinely novel audits.

Options considered: - **A.** Curriculum-ordering — replicate Wu et al. (2021)'s vision-domain confound in an LM setting. - **B.** MoE-routing — replicate Roller et al. (2021)'s fixed-hash-routing result.

Choice: A.

Reason: A reuses existing English-track infrastructure (tokenizer, corpus, perplexity eval) with zero new architecture. B would require standing up a routing architecture from scratch — real engineering cost the compute-rescoped plan can't absorb. A's three-condition mapping (difficulty-sorted / stage-matched-random / fully-scrambled, with total token budget matched across all three to avoid reproducing Wu et al.'s own dataset-growth confound) also fits the existing battery format without modification.

Consequences: If A's known vision-domain result doesn't transfer to the LM setting, that's genuinely ambiguous between "the battery has a problem" and "the confound is domain-specific" — flagged as a real interpretive risk in the experiment design, not resolvable by the choice of mechanism alone.

Revisit when: The user/team explicitly confirms or overrides this choice — currently the specific blocker keeping this and ADR-007 at Proposed rather than Accepted.

ADR-007 — Core mechanism list: 4 (GATE-ML, RigL, GrokFast, curriculum-ordering), not 6

Status: Proposed — same confirmation dependency as ADR-006.

Context: Six candidate mechanisms were identified during literature review (GATE-ML, RigL, GrokFast, DoReMi, CONGRAD, Localized Gradient Conflict Resolution). Compute rescoping (ADR-002) forced a cut.

Options considered: - **A.** All 6. - **B.** The 3 compute-feasible survivors of the original triage (GATE-ML, RigL, GrokFast), no positive control. - **C.** Those 3 plus curriculum-ordering as a dedicated 4th positive-control mechanism.

Choice: C.

Reason: A reintroduces DoReMi, CONGRAD, and Localized-Conflict, each of which has a reason beyond compute alone to stay deprioritized (DoReMi's multi-domain corpus-engineering cost is untouched by its code-availability downgrade; CONGRAD is a preference-alignment mechanism with no toy-scale pretraining analogue; Localized-Conflict's published base models are all 4B-8B with no toy-scale precedent). B leaves the battery with no known-answer calibration case, a real gap the novelty verification flagged directly. C adds the cheapest possible 4th mechanism specifically to close it.

Consequences: N drops to 3 for the headline effect-size correlation (curriculum-ordering is deliberately excluded from that specific analysis — its role is calibration, not audit) — a real statistical-power risk, and a possible-degeneracy risk if all three land on the same side of pass/fail, both documented explicitly rather than hidden.

Revisit when: Same trigger as ADR-006, plus: if the 4-mechanism core finishes with real compute runway left, DoReMi/CONGRAD/Localized-Conflict become live stretch-addition candidates per the original triage.

ADR-008 — Data-provenance reporting gate: hard-coded exclusion, not a documented convention

Status: Accepted

Context: A separate implementation session's NumPy/synthetic-data pipeline was drifting toward becoming the reported pipeline, and this was only caught through direct questioning, not any automated check — despite the intended scope (real data, real hardware) having been documented the entire time.

Options considered: - **A.** Document that synthetic-sourced results shouldn't be reported, and rely on the team following that. - **B.** A code-enforced gate in `orchestration/report.py` that filters out (and refuses to silently proceed past) any synthetic-tagged result.

Choice: B.

Reason: A already failed once — the documented intent existed the whole time the drift was happening, and it wasn't sufficient on its own. B makes the mistake structurally harder to repeat, consistent with the same reasoning already applied elsewhere (the C3 correlation's result type has no `p_value` field at all, rather than a convention not to report

one; the AuditBench pre-flight gate blocks stale validation rather than trusting it by default).

Consequences: Every `BatteryResult` and every log line must carry an explicit `data_source` tag — a small, non-optional addition to every module that touches results.

Revisit when: Not applicable — this is a permanent safeguard, not tied to a specific future trigger.

ADR-009 — Logging: local JSONL as source of truth, external dashboard optional

Status: Accepted

Context: Training runs happen on free-tier Colab/Kaggle sessions that can lose connectivity or be killed without warning, alongside a personal laptop with no guaranteed uptime either.

Options considered: - **A.** An external experiment-tracking service (W&B or similar) as the primary record. - **B.** Local JSONL as the primary record, with an optional external mirror.

Choice: B.

Reason: A risks losing an entire run's record if the service is unreachable at the exact moment a free-tier session dies — precisely the failure mode this environment is most exposed to. B guarantees a run is fully reconstructible from disk regardless of network state at any given moment; a mirror is additive, never load-bearing.

Consequences: No built-in web dashboard for monitoring runs in progress — the team relies on local log files and `viz/`'s static plots instead of live remote monitoring.

Revisit when: The team gains access to more reliable, persistent compute (e.g., a lab machine that isn't subject to session kills) where the downside of a remote-primary approach no longer applies.